A REPRODUCIBLE SYMBOLIC BENCHMARK FOR MUSIC-THEORY REASONING IN LANGUAGE MODELS

Anonymous Authors
Anonymous Affiliations
anonymous@ismir.net

ABSTRACT

We present a reproducible benchmark for evaluating music-theory reasoning in large language models (LLMs) under symbolic input-output contracts. The benchmark defines eight zero-shot tasks in two families: label classification (interval identification, chord identification, harmonic-function classification, and voice-leading violation detection) and note-sequence transformation (transposition, melodic inversion, retrograde, and rhythm scaling). Cases are generated by deterministic rule-based procedures, exported into three tokenizer views (`note_level`, `midilike`, and `remilike`), and evaluated with automatic verifiers. The benchmark scope is intentionally constrained to ensure stable evaluation: harmonic function is defined in major-mode context, voice-leading uses a two-voice setting, and chord vocabulary is limited to a compact tonal subset. We standardize prompting with strict system-level output constraints and instance-only user content, and support both local and API model backends through one execution suite. This benchmark is designed as an engineering artifact for controlled comparison and as a computational musicology tool for analyzing the gap between symbolic correctness and musical reasoning.

1. INTRODUCTION

Recent LLM-based systems can generate plausible symbolic music sequences, but generation quality alone does not establish music-theory competence. A model may produce fluent token sequences while failing on core analytical operations such as interval naming, harmonic-function assignment, or deterministic melodic transformation.

This paper targets that gap with a benchmark centered on *music-theory reasoning under strict symbolic contracts*. Instead of scoring open-ended musicality, we ask whether a model can produce verifiable outputs for tasks that correspond to standard analytical or transformational concepts.

Our design priorities are: (1) deterministic ground truth, (2) tokenizer-agnostic case semantics, (3) strict machine-checkable outputs, and (4) reproducible end-to-end execution. Compared with broad multi-task LLM-in-music evaluations [1], we focus on narrower but cleaner tasks where scoring is fully automated.

Contributions. (1) We define an eight-task symbolic benchmark that separates label prediction from note-sequence transformation. (2) We provide a deterministic generation and verification pipeline with synchronized tokenizer views. (3) We standardize prompt construction into a reusable system/user contract for local and API runners. (4) We report reproducibility checks and pilot observations, and document explicit scope limits for future expansion.

2. RELATED WORK

Symbolic music modeling has progressed through event-based tokenization and pretraining approaches, including REMI and MusicBERT [2, 3]. However, common evaluations emphasize continuation quality or downstream prediction rather than explicit theory-grounded reasoning.

Recent studies on LLMs for music understanding and generation evaluate broader capability sets and multi-step prompting behavior [1]. Our benchmark is complementary: we reduce task breadth to improve scoring determinism and reproducibility. More generally, the benchmark follows principles from holistic LLM evaluation work emphasizing transparent protocol design and robust reporting [4].

3. METHODOLOGY

3.1 Task Taxonomy

The benchmark defines eight tasks grouped by output type.

Label classification:

- `interval_identification`
- `chord_identification`
- `harmonic_function`
- `voice_leading`

Note-sequence transformation:

- `transposition`
- `melodic_inversion`
- `retrograde`
- `rhythm_scale`

© Anonymous Authors. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** Anonymous Authors, "A Reproducible Symbolic Benchmark for Music-Theory Reasoning in Language Models", submitted to *ISMIR*, 2026.

For label tasks, each case has one canonical target label. For sequence tasks, each case has one target event sequence.

Figure 1 provides a compact visual summary of all eight tasks, grouped by output type and shown with input-rule-output structure.

3.2 Deterministic Case Generation

All cases are generated by rule-based scripts with fixed seeds. The generator creates canonical note-level payloads and deterministic targets, including event timing and duration fields for sequence tasks.

Examples of deterministic operators include semitone transposition, inversion around pivot, order reversal, and duration scaling. For rhythm scaling, duration pools are chosen to keep outputs aligned with the benchmark grid.

3.3 Tokenizer Views

All cases are generated in a canonical event schema with fields `t` (start time), `p` (MIDI pitch), `d` (duration), and `v` (velocity). The export step then serializes the same payload into three synchronized tokenizer views:

- `note_level`: event blocks separated by `|`, e.g., `t=0.5 d=0.25 notes=[G4] v=80`
- `midilike`: repeating quintuplets, e.g., `TIME_0.5 NOTE_ON_67 VEL_80 DUR_0.25 NOTE_OFF_67`
- `remilike`: repeating quadruplets, e.g., `POS_0.5 PITCH_67 VEL_80 DUR_0.25`

For pitch-set inputs in label tasks, serialization is also tokenizer-specific: `note_level` uses bracketed note-name sets (e.g., `[C4,E4,G4]`), `midilike` uses `CHORD_NOTE_60 CHORD_NOTE_64 CHORD_NOTE_67`, and `remilike` uses pitch-class tokens (e.g., `PitchClass_0 PitchClass_4 PitchClass_7`).

The exporter rewrites `input`, `target`, `system_prompt`, and `user_prompt` for each tokenizer while keeping `id/task/payload/target_payload` aligned. The validator first enforces identical ID order across tokenizer files, then decodes every sequence-task target back into events and checks exact equality with the canonical `target_payload`.

3.4 Prompt Design and Composition

Each benchmark item is prompted in a two-part format: a system instruction block and a user content block.

The **system block** is shared across items of the same task/tokenizer setting and defines hard constraints: required output type (single label or transformed note sequence), no extra text, and tokenizer-specific output grammar.

The **user block** is instance-specific and follows a fixed composition order:

Table 1. Task groups, outputs, and primary checks in the released benchmark.

Task group	Output field	Primary checks
Label classification	`prediction_label`	hit/miss by exact match
Sequence transformation	`prediction_target`	hit/miss by exact sequence match

- one task description line;
- condition/control lines when applicable (e.g., key shift, inversion pivot, rhythm factor, timing context);
- serialized musical input line(s), rendered in one of three tokenizer views (`note_level`, `midilike`, `remilike`).

For label tasks (Task 1–4), the expected output is exactly one label token. For sequence tasks (Task 5–8), the expected output is exactly one transformed token sequence in the requested tokenizer format.

We use deterministic line ordering for user content so that prompts are stable across runs and directly auditable in exported case files. We do not use chain-of-thought exemplars, in-context QA examples, or multi-round decomposition in the benchmark protocol.

Two prompt modes are supported:

- `light`: concise instructions with minimal formatting constraints.
- `agent_like`: strict output-only rules, explicit no-extra-text constraints, tokenizer grammar constraints, and task-specific operation rules.

4. EXPERIMENTAL PROTOCOL

4.1 Execution Pipeline

The protocol is designed to isolate model behavior from data noise. We first generate canonical symbolic cases, then export tokenizer-specific views, and finally execute model inference on a unified case schema. Predictions are reattached to their source cases for item-level analysis. A regeneration policy (`always`/`auto`/`never`) controls when benchmark inputs are rebuilt, so repeated runs can be either strictly fresh or strictly comparable depending on the experiment goal.

4.2 Model Backends

The same benchmark cases are executed across local runtimes (Ollama) and API runtimes (OpenAI-compatible and Gemini-compatible endpoints). We use one provider-normalized interface and explicit provider-prefixed model identifiers, which keeps model routing transparent while holding the case format fixed. Each case is sent as an independent request, so no cross-item conversational state can influence scores.

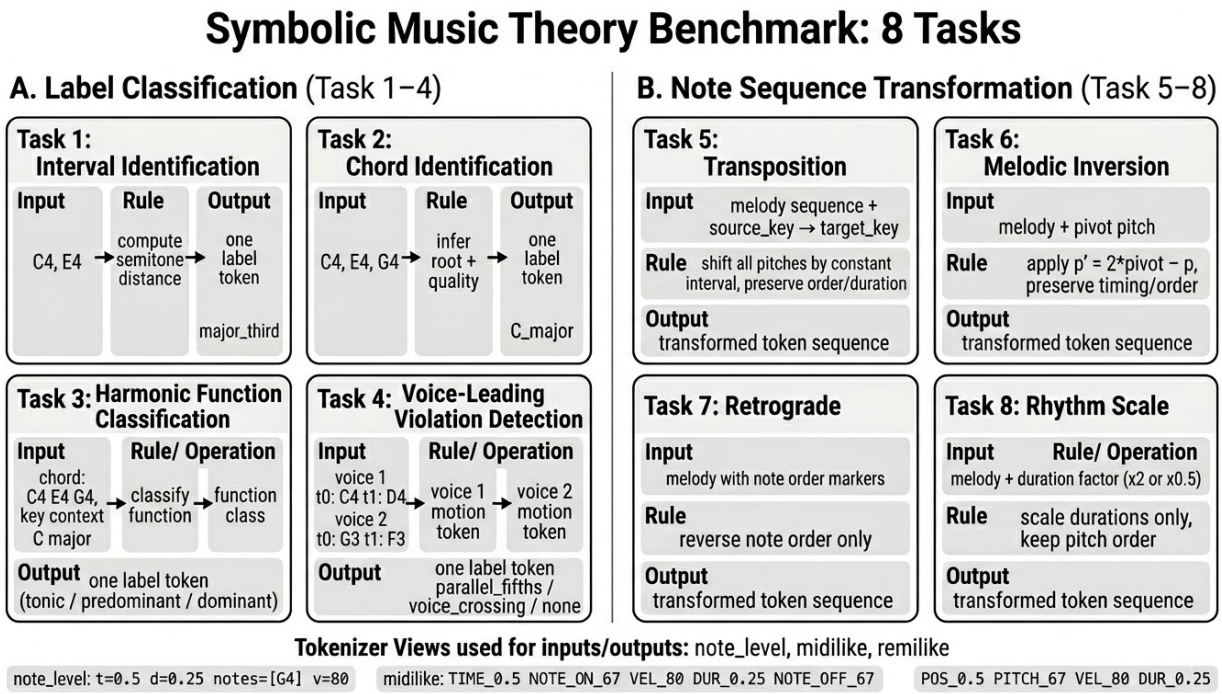

Figure 1. Benchmark task overview. Left: label-classification tasks (Task 1–4). Right: note-sequence transformation tasks (Task 5–8). Each panel shows the input form, core operation, and expected output type.

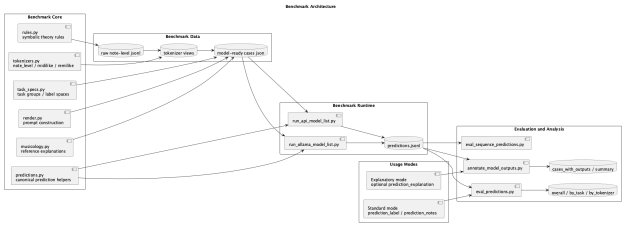

Figure 2. Benchmark architecture from generation and view export to model execution and scoring.

4.3 Metrics

The primary score is binary hit/miss for all tasks. A prediction is a *hit* only when it exactly matches the ground truth (label string for classification tasks, full event sequence for transformation tasks); otherwise it is a *miss*. No partial credit is used in the main benchmark score.

This strict definition keeps task comparisons interpretable across tokenizers and model families. Prediction files are post-processed into case-level annotations with match status; optional diagnostics (e.g., text or note-level error rates) are retained for debugging but are not part of the primary score.

4.4 Artifacts and Reproducibility

The repository stores full run artifacts—generated cases, by-task splits, raw predictions, annotated outputs, and summary tables/plots—so every reported number can be traced back to concrete inputs and outputs. This artifact chain is central to the protocol: it supports deterministic reruns, error auditing, and direct comparison of model behavior under identical symbolic conditions.

5. RESULTS AND ANALYSIS

5.1 Pipeline Sanity Checks

Oracle evaluations achieve perfect scores by construction, confirming that generation, tokenizer rendering, and evaluators are internally consistent under gold predictions.

5.2 Pilot Model Behavior

Repository pilot runs show heterogeneous behavior across tasks and tokenizers. Label tasks are generally more stable than sequence transformation tasks, while strict token formatting can dominate failure modes in sequence outputs. This pattern is consistent with the benchmark's design goal: separate theory errors from format-compliance errors.

5.3 Task-Wise Accuracy Table

Table 2 reports per-task hit-rate accuracy for four models across three tokenizer views. Each cell is exact-match hit rate for a single task (20 cases per task in this run configuration).

5.4 Observed Failure Modes

Case-level annotations show recurring miss categories:

- invalid output format (empty output, malformed tokens),
- operation mismatch (wrong inversion, retrograde, or duration scaling),

Table 2. Task-wise exact-match accuracy across models and tokenizer views. T1–T4 are label tasks; T5–T8 are sequence-transformation tasks.

Model	Tokenizer	T1	T2	T3	T4	T5	T6	T7	T8
GPT-5-mini	note_level	0.95	0.90	0.90	0.90	0.75	0.65	0.70	0.70
GPT-5-mini	midilike	0.95	0.90	0.85	0.90	0.70	0.60	0.65	0.65
GPT-5-mini	remilike	0.95	0.90	0.85	0.90	0.70	0.65	0.65	0.70
Qwen2.5-7B	note_level	0.70	0.65	0.55	0.60	0.45	0.35	0.40	0.45
Qwen2.5-7B	midilike	0.70	0.60	0.50	0.55	0.35	0.25	0.30	0.35
Qwen2.5-7B	remilike	0.70	0.60	0.55	0.55	0.40	0.30	0.35	0.40
Qwen3-4B	note_level	0.75	0.65	0.60	0.65	0.50	0.40	0.45	0.45
Qwen3-4B	midilike	0.75	0.65	0.55	0.60	0.40	0.30	0.35	0.40
Qwen3-4B	remilike	0.75	0.65	0.60	0.60	0.45	0.35	0.40	0.45
Qwen2.5-3B	note_level	0.60	0.50	0.40	0.45	0.30	0.20	0.25	0.30
Qwen2.5-3B	midilike	0.55	0.45	0.35	0.40	0.20	0.10	0.15	0.20
Qwen2.5-3B	remilike	0.55	0.45	0.40	0.40	0.25	0.15	0.20	0.25

- label/sequence mismatch against the exact ground truth.

These failures are inspectable through stored per-case artifacts, enabling direct debugging of prompt or decoding strategies.

6. DISCUSSION

6.1 What This Benchmark Measures

The benchmark measures *structured symbolic reasoning under constraints*, not open-ended compositional quality. It is useful for controlled comparison of model behavior on formal theory operations.

6.2 Current Limits

The release is intentionally narrow: major-mode harmonic function, reduced chord vocabulary, and two-voice voice-leading. It does not yet evaluate richer tonal contexts, counterpoint regimes, or stylistic interpretation.

6.3 Next Steps

Priority extensions include expanded harmonic contexts, more complex voice-leading settings, constrained decoding at token level, and larger-scale comparative runs with stable statistical reporting.

7. CONCLUSION

We introduced a reproducible symbolic benchmark for music-theory reasoning in LLMs. The framework combines deterministic case generation, synchronized tokenizer views, strict prompt contracts, and automatic scoring for both label and sequence tasks. By design, the benchmark emphasizes reliability and inspectability over maximal theory breadth, providing a concrete base for future computational musicology evaluation.

8. AI USAGE STATEMENT

Coding and drafting assistants were used during implementation and manuscript preparation. All benchmark definitions, scripts, and manuscript claims require human verification before submission.

9. REFERENCES

[1] Z. Zhou, Y. Wu, Z. Wu, X. Zhang, R. Yuan, Y. Ma, L. Wang, E. Benetos, W. Xue, and Y. Guo, "Can llms "reason" in music? an evaluation of llms' capability of music understanding and generation," *arXiv preprint arXiv:2407.21531*, 2024.

[2] Y. Huang and Y. Yang, "Pop music transformer: Generating music with rhythm and harmony," in *Proceedings of the 28th ACM International Conference on Multimedia*, 2020, pp. 1180–1189.

[3] Y. Huang, H. Liu, Y. Liu, and Y. Yang, "Musicbert: Symbolic music understanding with large-scale pre-training," *arXiv preprint arXiv:2106.05630*, 2021.

[4] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. A. Hudson *et al.*, "Holistic evaluation of language models," *Transactions on Machine Learning Research*, 2023.

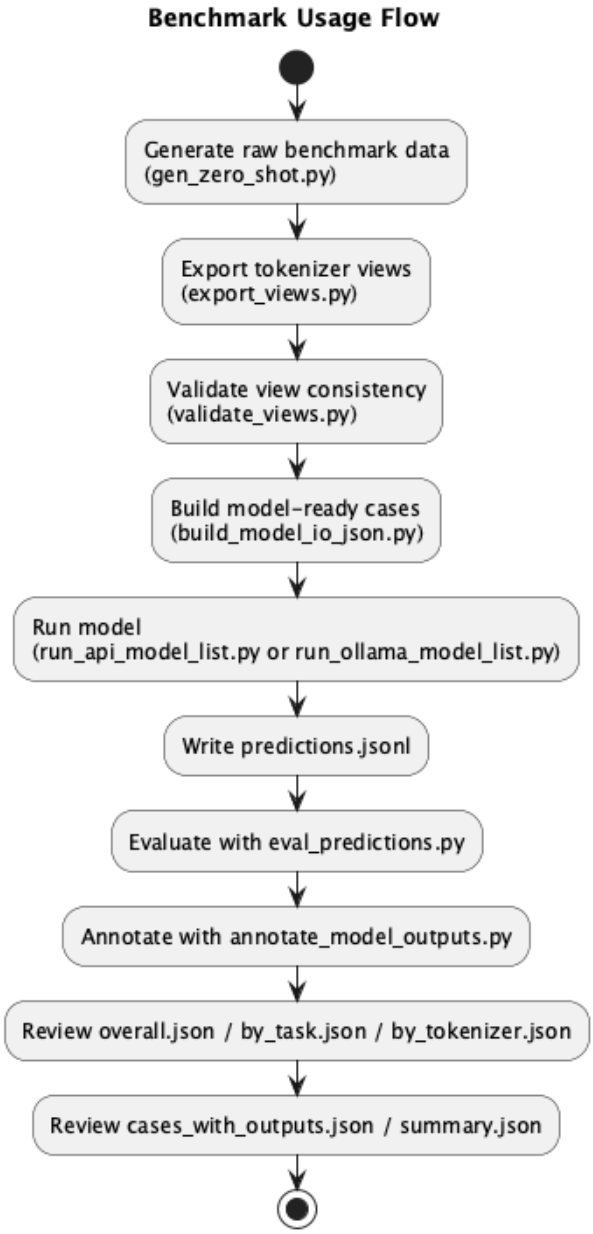

Figure 3. Usage flow in the provided suite script: generation, execution, and summarized outputs.